Honorables Class Design Handbook

Puffish Skills, GameStages, and KubeJS Design Standard

Working reference for class roots, base branches, subclasses, mastery, mod-aware icons, universal traits, and node IDs

---

Purpose. This handbook defines the standard layout and naming rules for all Honorables class skill trees. The goal is to keep every class consistent while still allowing each class, branch, subclass, and mastery path to feel distinct. This revision also standardizes universal derived traits, icon usage, definition syntax, and the final implementation checklist.

1 Core System Philosophy

The skill system is split across three layers:

System	Responsibility
Puffish Skills	Visual tree, skill nodes, attribute rewards, item/effect rewards, and command rewards.
GameStages	Progression flags, subclass unlocks, mastery unlocks, and KubeJS-readable permissions.
KubeJS	Runtime logic, universal derived traits, cooldowns, armor and weapon restrictions, ability behavior, and persistent player data.

1.1 Important Design Separation

Puffish Skills should not directly manage custom traits such as strength, endurance, constitution, perception, or resolve. Traits are derived by KubeJS from stages, attributes, and persistent player data.

> **Rule:** Puffish grants attributes and GameStages. KubeJS interprets those unlocks and attributes into gameplay traits.

1.2 Universal Design Rule

The entire progression system should follow this pipeline:

```
Puffish Skills
    -> Attributes + Commands
    -> GameStages
    -> KubeJS Trait Calculation
    -> Gameplay Logic
```

Classes never directly grant traits. Traits are always derived from attributes, stages, and persistent player data. Gameplay systems should prefer trait requirements over hard class requirements whenever practical.

2 Standard Class Tree Structure

Every class should follow the same high-level structure:

```
[class]_root
  -> branch_1 tiers 1-7
  -> branch_2 tiers 1-7
  -> branch_3 tiers 1-7
  -> subclass unlocks
  -> subclass mixed nodes
  -> mastery node
```

2.1 Root Node

Each class has one default unlocked root skill. This root node comes with the class and should not require purchase or unlock conditions.

Field	Standard
ID format	`[className]_root`
Icon	`minecraft:nether_star`
Unlock behavior	Default unlocked with the class
Purpose	Applies starting class attributes and establishes class identity
Example	`warrior_root`

2.2 Base Branches

Each class has three base branches from the root. All three branches eventually connect to all three subclass unlocks. The base branches shape the character, but they do not hard-lock subclass selection.

Current standard: 7 tiers per base branch. This must stay consistent across all branches and classes once finalized.

2.3 Branch ID Format

```
[className]_[branchName]_[tier]
```

Examples:

```
warrior_offense_1
warrior_offense_2
warrior_defense_1
warrior_utility_7
miner_extraction_3
mage_spellcasting_5
```

2.4 Branch Layout Rules

- Each class has exactly 3 base branches.
- Each branch has exactly 7 tiers for the current design standard.
- Branches are strictly linear unless intentionally shaped into a diamond pattern.
- Each branch should have 2-5 positive attributes.
- Each branch may have 0-2 negative attributes, but penalties should be handled cautiously.

- Negative scaling should increase class specialization, not subclass specialization.

3 Branch Penalty Standard

The safest penalty model is branch milestones, not a penalty on every single tier. Milestones are represented in naming and balance logic, not as separate explicit nodes.

3.1 Why Use Milestones?

If every tier adds a penalty, cumulative debuffs can accidentally erase other branch benefits or punish hybrid play too strongly. Milestone penalties are easier to tune and keep build flexibility alive.

3.2 Recommended Milestone Pattern

Tier	Positive Growth	Penalty Guidance
1	Small branch bonus	No penalty
2	Small branch bonus	No penalty or very light penalty
3	Moderate branch bonus	Optional first milestone penalty
4	Moderate branch bonus	No new penalty unless needed
5	Strong branch bonus	Optional second milestone penalty
6	Strong branch bonus	No new penalty unless needed
7	Capstone branch bonus	Optional final branch commitment penalty

3.3 Recommended Penalty Scale

Commitment Level	Suggested Penalty
Early branch investment	0% to -1%
Mid branch investment	-2% to -4%
Full branch investment	-5% to -8%
Extreme specialization	Avoid unless heavily tested

Rule: Subclass specialization should be enforced positively, not negatively. Penalties belong mostly to broad class identity and base branch commitment.

4 Warrior Branch Example

The Warrior demonstrates the base branch model with three branches: offense, defense, and utility.

4.1 Offense Branch

Focus	Attributes

Attack speed	`generic.attack_speed`
Attack damage	melee damage or sword damage attribute
Critical damage	`attributeslib:crit_damage`
Movement speed	`generic.movement_speed` or sprinting speed
Stamina	`puffish_attributes:stamina`

4.2 Defense Branch

Focus	Attributes
Armor	`generic.armor`
Regeneration	regeneration effect or healing / natural regeneration attribute
Max health	`generic.max_health`
Block absorption	KubeJS trait or shield-related stage logic

4.3 Utility Branch

Focus	Attributes or Systems
Kill XP	experience or KubeJS kill tracking
Attack range	reach-related attribute if available
Mana regen	`irons_spellbooks:mana_regen`
Armor access trait	KubeJS-derived endurance / strength trait

5 Subclass Unlocks

Subclass unlock nodes come after the base branch tiers. A player may be allowed to invest in multiple subclasses depending on server balance needs.

5.1 Subclass Unlock ID Format

```
[className]_[subclassName]_unlock
```

Examples:

```
warrior_knight_unlock
warrior_berserker_unlock
warrior_paladin_unlock
mage_spellsword_unlock
explorer_land_surveyor_unlock
```

5.2 Subclass GameStage Format

The preferred GameStage format is class-specific:

```
[className]_[subclassName]
```

Examples:

```
warrior_knight
warrior_berserker
warrior_paladin
mage_spellsword
miner_engineer
```

5.3 Subclass Unlock Behavior

A subclass unlock node should:

- Grant a minor related buff.
- Open the related GameStage.
- Use the subclass icon assigned in the icon guide.
- Avoid applying penalties.

```
{
  "type": "command",
  "data": {
    "command": "gamestage add @s warrior_knight"
  }
}
```

6 Subclass Node Structure

After the subclass is selected, the subclass branch contains 7 nodes before mastery.

6.1 Subclass Node Composition

Node Type	Count
Mixed ability nodes	5
Pure attribute nodes	2
Total subclass nodes	7
Mastery node	1 additional final node

6.2 Subclass Node ID Format

```
[className]_[subclassName]_[tier]
```

Tier numbering resets after the subclass unlock.
Examples:

```
warrior_knight_1
warrior_knight_2
warrior_knight_7
warrior_paladin_1
mage_spellsword_1
```

6.3 Subclass Design Rules

- Subclass nodes should specialize positively.
- Do not use subclass nodes to stack further penalties.
- Use mixed nodes for abilities, stage unlocks, KubeJS triggers, or minor attribute improvements.
- Use pure attribute nodes as pacing between ability nodes.
- Keep subclass identity clear and readable.

7 Mastery Nodes

Mastery nodes come after all nodes in a specific subclass branch have been unlocked.

7.1 Mastery Node ID Format

```
[className]_[subclassName]_mastery
```

Examples:

```
warrior_knight_mastery
warrior_berserker_mastery
warrior_paladin_mastery
explorer_land_surveyor_mastery
```

7.2 Mastery GameStage Format

```
[className]_[subclassName]_mastery
```

The node and GameStage may use the same ID for clarity.

7.3 Mastery Design Rule

Mastery should not primarily grant large raw stat bonuses. Mastery should grant power skills, ability upgrades, or enhancements to previous skills.

Examples:

- Shield Bash gains a stun.
- Paladin healing affects nearby allies.
- Prospector reveals larger ore vein areas.
- Land Surveyor identifies settlement quality or nearby resource density.
- Spellsword gains a special weapon imbue interaction.

8 Universal Derived Traits

Traits are not Puffish Skills rewards. Traits are calculated by KubeJS based on attributes, GameStages, and persistent player data. Traits should be universal RPG-style character capabilities, not class-specific labels.

8.1 Trait Calculation Pattern

```
attributes + stages + player persistent data
  -> KubeJS trait calculation
  -> universal trait score
  -> gameplay effect
```

8.2 Design Philosophy

Traits should never reference a player's class directly. A player arrives at their trait values through their build rather than through arbitrary class assignment. This allows equipment restrictions, environmental systems, and ability requirements to be based on character development instead of hard class locks.

> **Rule:** Classes influence traits indirectly through attributes. KubeJS derives trait values from those attributes and applies gameplay logic.

8.3 Strength

Field	Value
Definition	Represents raw physical power and the ability to exert force.
Derived From	Melee Damage; Sword Damage; Axe Damage; Max Health; Attack Damage modifiers.
Gameplay Uses	Heavy weapon requirements; greatsword and hammer effectiveness; physical interaction checks; knockback generation; future carrying capacity systems; object interaction requirements.
Core Question	How much force can this character produce?

8.4 Endurance

Field	Value
Definition	Represents sustained physical capability and the body's ability to continue performing under stress.
Derived From	Max Health; Armor; Armor Toughness; Stamina; Resistance as a minor contribution; movement-related attributes.

Gameplay Uses	Heavy armor requirements; heavy shield requirements; heavy weapon requirements in conjunction with Strength; fall damage reduction; saturation efficiency; hunger depletion reduction; sprint duration; stamina regeneration; encumbrance penalties; long-distance travel efficiency; physical exhaustion resistance; extended combat performance.
Core Question	How long can this character keep going?

8.5 Constitution

Field	Value
Definition	Represents biological health, recovery, and resistance to internal and external harm. Unlike Endurance, Constitution measures bodily resilience rather than work capacity.
Derived From	Healing; Natural Regeneration; Magic Resistance; Resistance; Healing Received; Max Health as a minor contribution.
Gameplay Uses	Legendary Survival Overhaul disease resistance; infection resistance; cold and heat resistance; bleeding, wound, and injury recovery; curse resistance; debuff duration reduction; magic resistance scaling; negative effect mitigation; healing, regeneration, and bandage effectiveness; recovery after combat; poison and wither resistance; future radiation or toxic environment systems.
Core Question	How healthy and resilient is this character's body?

8.6 Dexterity

Field	Value
Definition	Represents precision, coordination, and fine motor skill. Dexterity governs technical execution rather than movement.
Derived From	Attack Speed; Draw Speed; Critical Chance; Reach; precision-related attributes.
Gameplay Uses	Bow handling; precision weapon handling; fine tool usage; future lockpicking; craftsmanship; spell accuracy; reload mechanics.
Core Question	How precisely can this character perform complex actions?

8.7 Agility

Field	Value
Definition	Represents full-body movement and athletic mobility. Agility should not overlap with Dexterity.
Derived From	Movement Speed; Sprint Speed; Jump Height; Swim Speed; Fall Reduction.

Gameplay Uses	Dodging; climbing; traversal; parkour; mobility abilities; dash mechanics; general movement capability.
Core Question	How well can this character move?

8.8 Intelligence

Field	Value
Definition	Represents technical knowledge, magical understanding, and learned expertise.
Derived From	Spell Power; Maximum Mana; Experience Gain; Cooldown Reduction; Cast Speed.
Gameplay Uses	Spell tier requirements; engineering systems; advanced crafting; enchanting; magical devices; research mechanics.
Core Question	How much has this character learned?

8.9 Wisdom

Field	Value
Definition	Represents practical understanding, intuition, and application of knowledge.
Derived From	Healing; Mana Regeneration; Luck; Experience; Regeneration.
Gameplay Uses	Healing effectiveness; herbalism; animal interaction; support abilities; nature-related systems; settlement guidance; decision-oriented mechanics.
Core Question	How well does this character understand the world?

8.10 Perception

Field	Value
Definition	Represents awareness and the ability to notice important information.
Derived From	Luck; Reach; Experience; survey-related bonuses.
Gameplay Uses	Structure discovery; ore detection; trap detection; hidden loot; resource surveying; enemy awareness; exploration systems.
Core Question	How much does this character notice?

8.11 Resolve

Field	Value

Definition	Represents mental fortitude and the ability to continue functioning despite hardship.
Derived From	Endurance; Stamina; Resistance; Mana Regeneration; Healing.
Gameplay Uses	Crowd control resistance; fear resistance; stun duration reduction; slow resistance; spell concentration; cast interruption resistance; buff maintenance; knockback recovery.
Core Question	How determined is this character under pressure?

8.12 Trait Relationship Summary

Trait	Core Question
Strength	How much force can this character produce?
Endurance	How long can this character keep going?
Constitution	How healthy and resilient is this character's body?
Dexterity	How precisely can this character perform complex actions?
Agility	How well can this character move?
Intelligence	How much has this character learned?
Wisdom	How well does this character understand the world?
Perception	How much does this character notice?
Resolve	How determined is this character under pressure?

8.13 Guiding Principles

#	Principle
1	Traits are calculated, never directly granted.
2	Classes influence traits indirectly through attributes.
3	Equipment restrictions should use trait thresholds instead of class checks whenever possible.
4	KubeJS is responsible for interpreting traits into gameplay effects.
5	Traits should remain universal so unconventional builds remain possible while naturally specializing through progression.

9 Icon Standards

Icons should create a consistent visual language across all six class trees. One icon should mean one thing whenever possible. If a node could reasonably use multiple icons, use the hierarchy below. Non-structural icons should prefer verified modpack item IDs when a good representative item exists.

9.1 Icon Hierarchy

Priority	Rule
1. Structural Icons	If a node is a class root, subclass unlock, ability unlock, trait unlock, or mastery node, use the reserved structural icon.
2. Ability Icons	If a node grants a specific active or passive ability, use an icon representing that ability.
3. Attribute Icons	If a node is purely statistical, use the canonical attribute icon.
4. Flavor Icons	Use class or theme flavor icons only when none of the above apply.

9.2 Reserved Structural Icons

These icons are reserved and should not be reused for unrelated purposes.

Node Purpose	Icon
Class Root	`minecraft:nether_star`
Mastery	`minecraft:dragon_egg`
Subclass Unlock	`minecraft:experience_bottle`
Generic Ability Unlock	`minecraft:amethyst_shard`
Trait Unlock	`minecraft:echo_shard`

9.3 Canonical Attribute Icon Dictionary

Every attribute or repeated concept should use the same icon across every class tree. These entries prefer verified items from the installed modpack when they clearly represent the concept.

Attribute or Concept	Canonical Icon
Melee damage	`simplyswords:iron_longsword`
Sword damage	`simplyswords:diamond_longsword`
Axe damage	`simplyswords:diamond_greataxe`
Attack speed	`better_weaponry:netherite_dagger`
Critical chance	`apotheosis:broadhead_arrow`
Critical damage	`spartanweaponry:diamond_heavy_crossbow`
Ranged damage	`iceandfire:dragonbone_bow`
Arrow damage	`apotheosis:explosive_arrow`
Draw speed	`spartanweaponry:bolt`
Armor	`immersiveengineering:armor_steel_chestplate`
Armor toughness	`iceandfire:dragonsteel_fire_chestplate`
Max health	`legendarysurvivaloverhaul:heart_container`
Resistance	`better_weaponry:amethyst_shield`
Melee resistance	`immersiveengineering:armor_steel_helmet`
Magic resistance	`irons_spellbooks:protection_rune`
Stamina	`arthys_rpg_arms:ring_of_stamina`
Movement speed	`legendarysurvivaloverhaul:desert_boots`
Sprinting speed	`iceandfire:amphithere_feather`

Jump	alexsmobs:kangaroo_hide
Fall reduction	alexsmobs:tarantula_hawk_wing
Swim speed	alexsmobs:flying_fish
Step height	immersiveengineering:steel_scaffolding_standard
Mining speed	immersiveengineering:drill
Pickaxe speed	immersiveengineering:pickaxe_steel
Breaking speed	arthys_rpg_arms:ebonite_pickaxe
Fortune	arthys_rpg_arms:emerald_pendant
Block reach	minecraft:spyglass
Healing	legendarysurvivaloverhaul:bandage
Natural regeneration	legendarysurvivaloverhaul:healing_herbs
Consuming speed	farmersdelight:skillet
Experience	quark:ancient_tome
Luck	apotheosis:potion_charm
Stealth	mowziesmobs:naga_fang_dagger
Tamed damage	alexsmobs:falconry_glove
Tamed resistance	alexsmobs:animal_dictionary
Spell power	irons_spellbooks:arcane_rune
Max mana	irons_spellbooks:mana_ring
Mana regeneration	irons_spellbooks:greater_conjurers_talisman
Cooldown reduction	irons_spellbooks:cooldown_rune
Cast time reduction	irons_spellbooks:cast_time_ring
Casting movement speed	irons_spellbooks:speed_boots
Holy spell power	irons_spellbooks:holy_rune
Fire spell power	irons_spellbooks:fire_rune
Ice spell power	irons_spellbooks:ice_rune
Lightning spell power	irons_spellbooks:lightning_rune
Nature spell power	irons_spellbooks:nature_rune
Ender spell power	irons_spellbooks:ender_rune
Blood spell power	irons_spellbooks:blood_rune
Eldritch spell power	irons_spellbooks:eldritch_manuscript
Evocation spell power	irons_spellbooks:evocation_rune

9.4 Canonical Subclass Icons

Subclass unlock icons should communicate subclass identity and should not overlap with structural icons.

Class	Subclass	Icon
Miner	Quarryman	immersiveengineering:bucket_wheel
Miner	Prospector	immersiveengineering:sample_drill
Miner	Engineer	immersiveengineering:hammer
Farmer	Rancher	alexsmobs:straddle_saddle
Farmer	Herbalist	regions_unexplored:flowering_shrub
Farmer	Brewer	irons_spellbooks:alchemist_cauldron
Explorer	Pathfinder	irons_spellbooks:wayward_compass

Explorer	Treasure Hunter	`aquaculture:treasure_chest`
Explorer	Land Surveyor	`immersiveengineering:survey_tools`
Warrior	Knight	`arthys_rpg_arms:guardian_armor_chestplate`
Warrior	Berserker	`born_in_chaos_v1:great_reaper_axe`
Warrior	Paladin	`irons_spellbooks:paladin_chestplate`
Ranger	Hunter	`spartanweaponry:diamond_longbow`
Ranger	Sharpshooter	`spartanweaponry:huge_arrow_quiver`
Ranger	Scout	`antiqueatlas:empty_antique_atlas`
Mage	Elementalist	`irons_spellbooks:cinderous_soul_rune`
Mage	Spellsword	`irons_spellbooks:amethyst_rapier`
Mage	Enchanter	`irons_spellbooks:arcane_anvil`

9.5 Ability Icon Examples

Ability Type	**Suggested Icon**
Shield Bash	`better_weaponry:amethyst_shield`
Heal	`legendarysurvivaloverhaul:bandage`
Blink	`irons_spellbooks:ender_rune`
Whirlwind	`simplyswords:diamond_greataxe`
Fire spell	`irons_spellbooks:fire_rune`
Ice spell	`irons_spellbooks:ice_rune`
Lightning spell	`irons_spellbooks:lightning_rune`
Ore detection	`immersiveengineering:coresample`
Survey	`immersiveengineering:survey_tools`
Animal skill	`alexsmobs:animal_dictionary`
Brewing skill	`irons_spellbooks:alchemist_cauldron`
Enchanting skill	`irons_spellbooks:arcane_anvil`

9.6 Icon Syntax

```
"icon": {
  "type": "item",
  "data": {
    "item": "simplyswords:iron_longsword"
  }
}
```

```
"icon": {
  "type": "effect",
  "data": {
    "effect": "minecraft:strength"
  }
}
```

```
"icon": {
  "type": "skill",
  "data": {
```

```
    "skill": "warrior_root"
  }
}
```

10 Definition Syntax Reference

10.1 General Definition Template

```
{
  "definition_id": {
    "title": "Skill Name",
    "icon": {
      "type": "item",
      "data": {
        "item": "simplyswords:iron_longsword"
      }
    },
    "description": [
      "Line 1 description",
      "Line 2 description"
    ],
    "rewards": [
      {
        "type": "attribute",
        "data": {
          "attribute": "generic.max_health",
          "value": 2.0,
          "operation": "addition"
        }
      },
      {
        "type": "command",
        "data": {
          "command": "gamestage add @s warrior_knight"
        }
      }
    ]
  }
}
```

10.2 Attribute Operations

Operation	Use Case
`addition`	Flat bonuses such as +2 health, +1 armor, +1 reach.
`multiply_base`	Percentage-style scaling against base value.
`multiply_total`	Late-game scaling against the total modified value. Use carefully.

10.3 Typical Reward Types

10.3.1 Attribute Node

```
{
  "type": "attribute",
  "data": {
    "attribute": "generic.max_health",
    "value": 2,
    "operation": "addition"
  }
}
```

10.3.2 Passive Effect

```
{
  "type": "effect",
  "data": {
    "effect": "regeneration",
    "amplifier": 0
  }
}
```

10.3.3 Unlock Stage

```
{
  "type": "command",
  "data": {
    "command": "gamestage add @s warrior_knight"
  }
}
```

10.3.4 Give Item

```
{
  "type": "item",
  "data": {
    "item": "irons_spellbooks:wimpy_spell_book"
  }
}
```

11 Recommended Node Types

Node Type	Example ID	Purpose
Root node	`warrior_root`	Default unlocked class identity node.
Core tier node	`warrior_offense_1`	Incremental branch progression.
Subclass unlock	`warrior_knight_unlock`	Opens class-specific subclass stage.

Subclass node	`warrior_knight_1`	Mixed ability and attribute progression after subclass choice.
Mastery node	`warrior_knight_mastery`	Unlocks mastery stage and major mechanic upgrade.

12 Quality-of-Life Trees

A separate non-class tree may be added for quality-of-life bonuses. This is optional.

12.1 Allowed QoL Themes

- Minor mining speed.
- Minor consumption speed.
- Minor hunger or stamina comfort.
- Small travel convenience.
- Small crafting or inventory convenience.

12.2 QoL Rule

Quality-of-life trees must not invalidate class identity.

> Good QoL node: +5% mining speed for everyone.
> Bad QoL node: +25% melee damage for everyone.

13 Design and Implementation Checklist

Before implementing a class tree, verify the following:

- The class has exactly one `[class]_root` node.
- The root node is default unlocked and uses `minecraft:nether_star`.
- The class has exactly 3 base branches.
- Each base branch uses 7 linear tiers unless an intentional diamond pattern is designed.
- Branch node IDs follow `[class]_[branch]_[tier]`.
- Subclass unlock IDs follow `[class]_[subclass]_unlock`.
- Subclass GameStages follow `[class]_[subclass]`.
- Subclass nodes follow `[class]_[subclass]_[tier]` and reset tier numbering to 1.
- Each subclass branch has 5 mixed nodes and 2 pure attribute nodes.
- Mastery nodes follow `[class]_[subclass]_mastery`.
- Mastery unlocks power skills or upgrades previous skills rather than simply adding large raw stats.
- Traits are calculated by KubeJS, not directly by Puffish Skills.
- Trait requirements are preferred over hard class requirements wherever practical.
- Negative scaling is handled through branch milestones, not every node.
- Subclass specialization is enforced positively, not through further penalties.
- Every node uses the correct ID format.
- Every node uses the correct canonical icon according to the icon hierarchy.

- Every node includes a clear title and description.
- Every node grants only approved reward types.
- Nodes use attributes instead of directly modifying traits.
- Branches follow their intended attribute themes.
- Subclass unlocks open the correct class-specific GameStage.
- Mastery enhances mechanics rather than primarily granting raw attributes.
- The design supports KubeJS trait derivation.
- Icon meanings do not overlap.
- Duplicate gameplay purposes are avoided.
- Class identity is preserved while allowing hybrid progression.
- Future KubeJS integration should not require redesigning the tree.